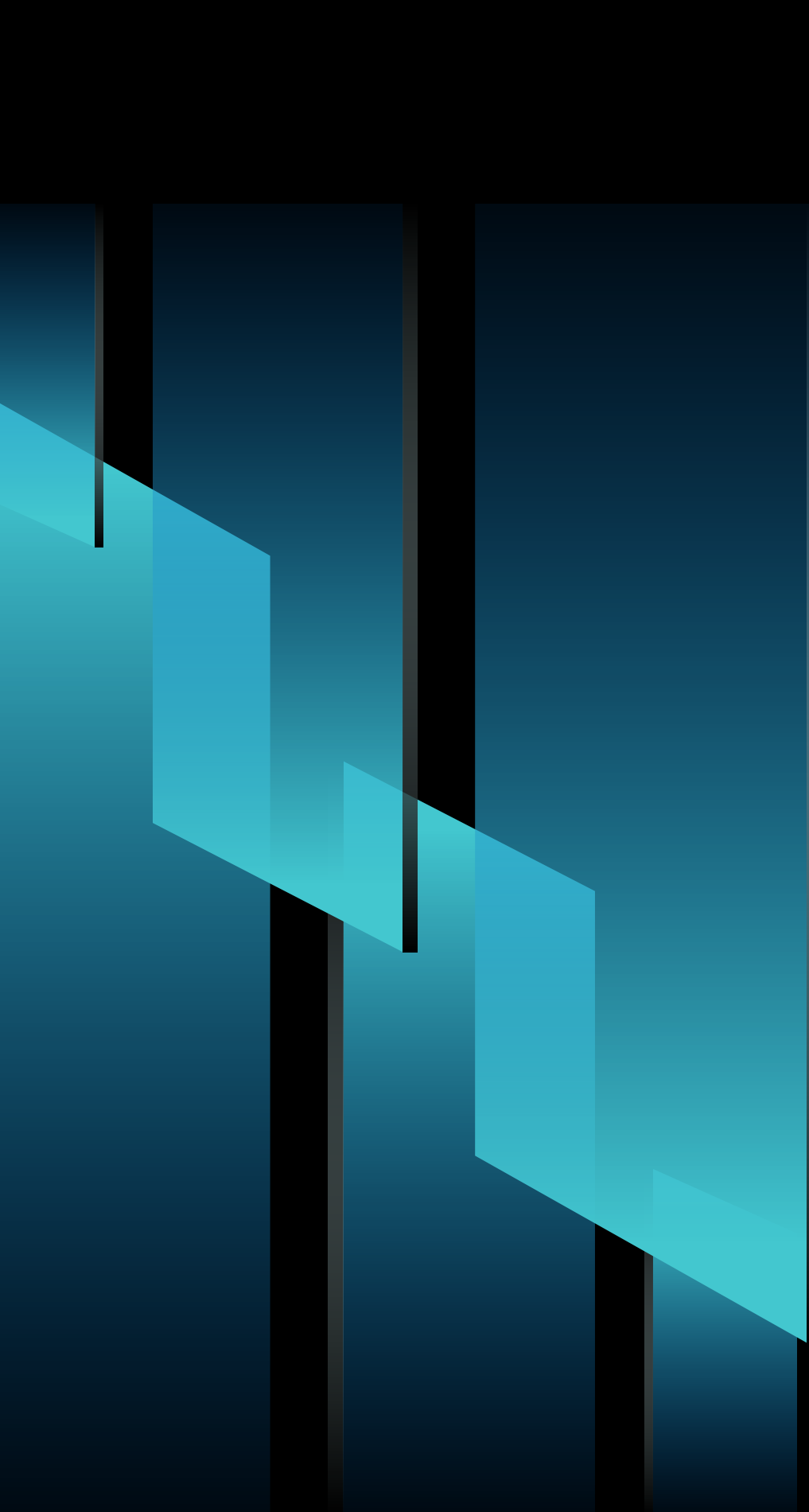




CURRENCY CONVERTER

JAVA PROJECT USING API

BY RIDDHIMAN DUTTA

REGISTRATION NO. -25BAI11325

INTRODUCTION

In today's interconnected global economy, real-time currency conversion is essential for international trade, travel, and online transactions. The Java Currency Converter is a lightweight, console-based utility program designed to fetch live exchange rate data via an external web API and compute accurate currency translations instantly. It bridges core object-oriented programming concepts with practical web connectivity, eliminating the need for heavy graphical user interfaces while maintaining professional accuracy through native internationalization packages.

PROBLEM STATEMENT

Manually tracking daily foreign exchange fluctuations or using bulky, bloated software tools is impractical for quick, on-the-fly calculations.

Furthermore, standard programming assignments often rely on hardcoded static exchange rates, making them obsolete immediately upon market changes. There is a need for a streamlined, lightweight utility tool that can interact with live web services securely, parse responses without external software dependencies, and render exact localized financial symbols dynamically.

FUNCTIONAL REQUIREMENTS

- Live Rate Fetching: The system must connect dynamically to an external web service API to pull up-to-date market rates.
- Menu Display: The application must display an initial text chart listing supported top global currency names and ISO codes.
- Input Validation: The system must accept user choices for base currency, target currency, and a valid numerical monetary amount.
- Conversion Computation: The program must process the target conversion pair and compute the correct final value.
- Formatted Output: The application must present the final converted output accompanied by the official target currency symbol (e.g., ₹, \$, €).

NON-FUNCTIONAL REQUIREMENTS

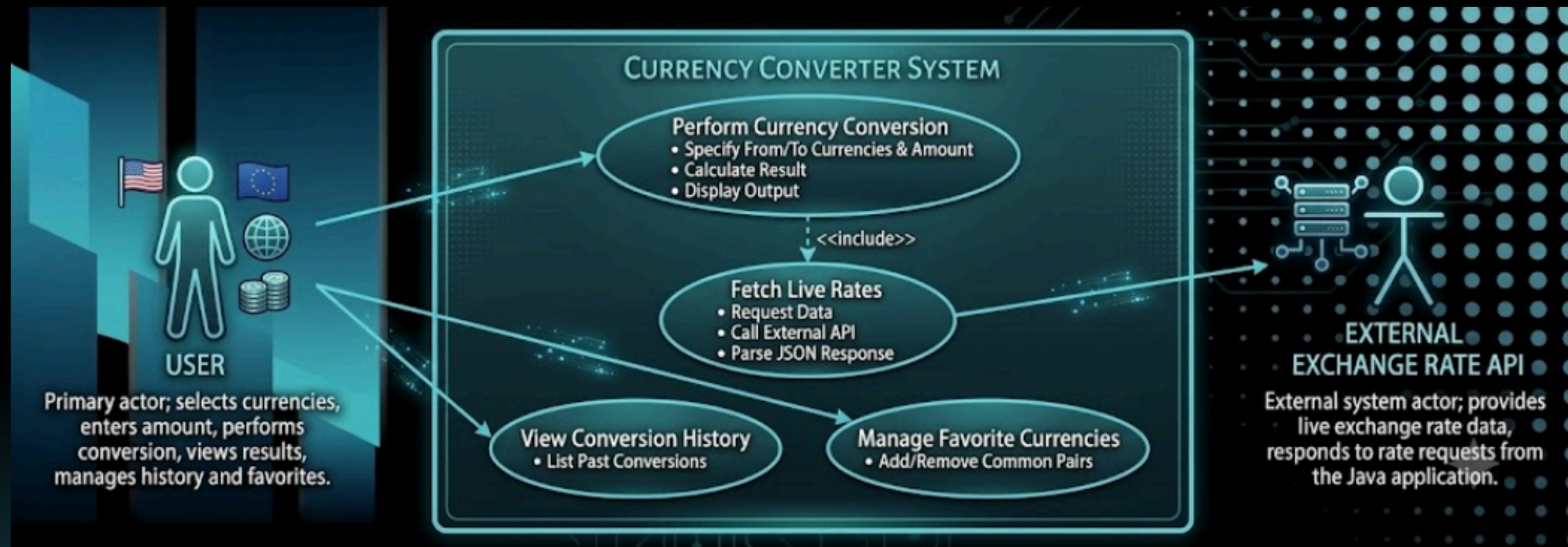
- Performance: The network request and conversion calculation must execute within a few seconds depending on internet latency.
- Reliability: The application must gracefully handle network disruptions, invalid currency codes, or HTTP error statuses without crashing.
- Maintainability: The codebase must remain lightweight, clean, and structured with clear separation between UI interaction and API logic.
- Portability: The program must execute smoothly on any machine with Java SE 11 or higher installed due to built-in native networking libraries.

SYSTEM ARCHITECTURE

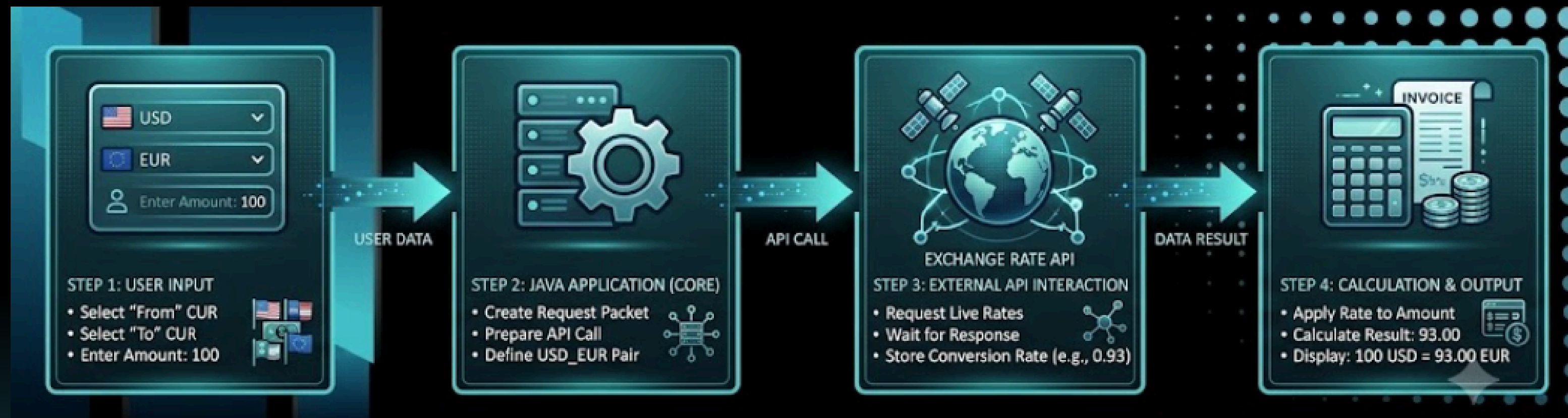
The application follows a streamlined client-server procedural architecture:

1. Presentation Layer (Console UI): Handles user input collection (Scanner) and final formatted console output (System.out.printf).
2. Service/Integration Layer (HttpClient): Constructs secure GET requests, dispatches HTTP queries to the ExchangeRate-API endpoint, and receives raw JSON text streams.
3. Data Parsing & Formatting Layer: Splits raw text string payloads to isolate target numbers, converts them to numerical types, and cross-references java.util.Currency for symbol mapping.

USE CASE DIAGRAM



SEQUENCE DIAGRAM



CLASS/COMPONENT DIAGRAM

- Class Name: currencyConverter
- Methods:
 - public static void main(String[] args)
 - public static double convertCurrency(String base, String target, double val)

DESIGN DECISIONS & RATIONALE

- Choice of Java HttpClient: Utilized native Java 11 HTTP tools instead of older legacy classes (URLConnection) or third-party frameworks to maintain modern, concise, and readable code.
- Avoiding Heavy JSON Parsers (Gson/Jackson): Decided to use standard string splitting (bodyText.split()) rather than requiring external jar file configurations. This keeps the project lightweight, single-file friendly, and easy to run for beginners.
- Integration of java.util.Currency: Used native Java localization libraries to automatically handle official currency symbols instead of manual hardcoding, ensuring professional-grade formatting standards.

IMPLEMENTATION DETAILS

The core logic resides in a single executable class file (`CurrencyConverter.java`), utilizing `java.net.http` to execute synchronous web requests to the API. Instead of relying on heavy external libraries, the program uses simple string manipulation to isolate the "conversion_result" attribute from the JSON response. Once extracted, the application leverages the native `java.util.Currency` class to format the final output with the correct international symbol (such as ₹ or €). Finally, `Scanner` handles console inputs, while standard try-catch blocks ensure that network timeouts and invalid responses are managed gracefully without crashing the program.

SCREENSHOTS

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.Currency;
import java.util.Scanner;

public class currencyConverter {
    Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Information Chart
        System.out.println(x: "|-----Currency Converter-----|");
        System.out.println(x: "1. US Dollar Code: USD");
        System.out.println(x: "2. Euro Code: EUR");
        System.out.println(x: "3. Japanese Yen Code: JPY");
        System.out.println(x: "4. British Pound Sterling Code: GBP");
        System.out.println(x: "5. Chinese Renminbi (Yuan) Code: CNY");
        System.out.println(x: "6. Swiss Franc Code: CHF");
        System.out.println(x: "7. Australian Dollar Code: AUD");
        System.out.println(x: "8. Canadian Dollar Code: CAD");
        System.out.println(x: "9. Hong Kong Dollar Code: HKD");
        System.out.println(x: "10. Indian Rupee Code: INR");
        System.out.println(x: "|-----|");

        // User Input
        System.out.print(s: "Select Currency to convert from: ");
        String fromCurrency = scanner.next().toUpperCase();

        System.out.print(s: "Select Currency to convert to: ");
        String toCurrency = scanner.next().toUpperCase();

        System.out.print(s: "Enter amount to convert: ");
        double amount = scanner.nextDouble();

        scanner.close();

        double finalVal = convertCurrency(fromCurrency, toCurrency, amount);

        if (finalVal != -1) {
            Currency cur = Currency.getInstance(toCurrency);
            System.out.printf(format: "Converted Amount: %s %.2f%n", cur.getSymbol(), finalVal);
        } else {
            System.out.println(x: "Conversion failed due to error.");
        }
    }
}
```

SCREENSHOTS

```
//Conversion Function
public static double convertCurrency(String base, String target, double val) {
    try {
        String myKey = "ea47c2f05d143f5737838ef6";
        String endpoint = "https://v6.exchangerate-api.com/v6/" + myKey + "/pair/" + base + "/" + target + "/" + val;

        HttpClient httpClient = HttpClient.newHttpClient();
        HttpRequest apiReq = HttpRequest.newBuilder()
            .uri(URI.create(endpoint))
            .GET()
            .build();

        HttpResponse<String> resp = httpClient.send(apiReq, HttpResponse.BodyHandlers.ofString());

        if (resp.statusCode() == 200) {
            String bodyText = resp.body();

            if (bodyText.contains(s: "conversion_result")) {
                String[] splitData = bodyText.split(regex: "\"conversion_result\":");
                String rawNumber = splitData[1].split(regex: "[,}"])[0].trim();
                return Double.parseDouble(rawNumber);
            } else {
                System.out.println("Server response issue: " + bodyText);
            }
        } else {
            System.out.println("Bad status code: " + resp.statusCode());
        }
    } catch (Exception err) {
        System.out.println("Something went wrong: " + err.getMessage());
    }

    return -1;
}
```

SCREENSHOTS

```
|-----Currency Converter-----|  
1. US Dollar Code: USD  
2. Euro Code: EUR  
3. Japanese Yen Code: JPY  
4. British Pound Sterling Code: GBP  
5. Chinese Renminbi (Yuan) Code: CNY  
6. Swiss Franc Code: CHF  
7. Australian Dollar Code: AUD  
8. Canadian Dollar Code: CAD  
9. Hong Kong Dollar Code: HKD  
10. Indian Rupee Code: INR  
Select Currency to convert from: inr  
Select Currency to convert to: usd  
Enter amount to convert: 500  
Converted Amount: $ 5.21
```

TESTING APPROACH

- Unit & Integration Testing: Tested individual currency pairs (e.g., USD to INR, EUR to GBP) to confirm mathematical accuracy against live market benchmarks.
- Boundary & Negative Testing:
- Inputting invalid 3-letter currency codes (e.g., XYZ) to verify that error handling blocks intercept bad status codes.
- Inputting negative values or text characters into numerical amount fields to check robust exception management.

CHALLENGES FACED

- **JSON Parsing Complexity:** Standard Java lacks native lightweight JSON mapping out-of-the-box, which initially risked requiring bulky external libraries. Resolution: Overcame this by designing a clean string-splitting algorithm that extracts the exact target floating-point value safely.
- **API Route Formatting:** Constructing precise REST API query parameters within the URL string required careful string concatenation to avoid malformed endpoint requests.

LEARNINGS & KEY TAKEAWAYS

- Gained hands-on experience integrating third-party web REST APIs directly inside a core Java program.
- Mastered modern Java networking components including HttpClient, HttpRequest, and HttpResponse.
- Understood the value of utilizing built-in localization libraries like `java.util.Currency` for professional financial formatting.

FUTURE ENHANCEMENTS

- GUI Integration: Transitioning the console application into a desktop interface using JavaFX or Swing.
- Historical Data Tracking: Adding local file logging (FileWriter) to record a user's past conversion history.
- Offline Fallback Caching: Implementing cached fallback rates in case the user experiences a network disconnect.

REFERENCES

- Official Java Documentation: `java.net.http` & `java.util.Currency` packages.
- ExchangeRate-API Documentation & Endpoints (<https://www.exchangerate-api.com/>).
- Core Java Programming Guidelines and Best Practices.

The background is a dark navy blue. On the left side, there are several overlapping, semi-transparent blue geometric shapes, including rectangles and parallelograms, creating a layered effect. On the right side, there is a grid of small, light blue dots that fade out towards the center.

THANK YOU